Median of Two Sorted Arrays: Optimal Logarithmic Partitioning

Anonymous Researcher

Abstract—This paper presents a formal analysis of the problem of finding the median of two sorted arrays. The task requires identifying the median element of the union of two independently sorted arrays, X and Y, with lengths m and n, respectively. While a naive merge operation yields a linear time complexity of O(m+n), this study demonstrates that a logarithmic time complexity of O(log(min(m, n))) is achievable through a binary search approach on the arrays' partition indices. The analysis formally proves the correctness of the partition invariants and evaluates the algorithm's temporal and spatial complexity characteristics.

1 Introduction

The identification of the median element across multiple sorted datasets is a foundational problem in computer science. Algorithms for median selection are frequently employed in distributed database queries, sensor network data aggregation, and robust statistical estimators. The fundamental challenge lies in locating the median without physically merging the datasets, which would incur a linear time cost.

This study analyzes an optimal algorithm that achieves logarithmic time complexity by leveraging the inherent sorted properties of the input arrays. Instead of examining individual elements, the algorithm searches for an optimal partition point that divides the combined dataset into two halves of equal length. This approach transforms the median selection problem into a constrained binary search problem. The objective is to establish the theoretical lower bound for this problem and demonstrate the correctness of the binary search methodology.

2 Problem Formulation

Let X and Y be two sorted arrays of lengths m and n, respectively. The elements of X and Y belong to a totally ordered domain. The combined dataset Z is defined as the multiset union of X and Y, having a total length of $m + n$. The median of Z is defined as the value that separates the lower half of Z from the upper half.

If $m + n$ is odd, the median is the unique middle element. If $m + n$ is even, the median is defined as the arithmetic mean of the two middle elements. The formal objective is to compute the median of Z using at most $O(\log(\min(m, n)))$ operations, utilizing strictly $O(1)$ auxiliary space.

3 Methodology: Binary Search on Partitions

The optimal solution operates by defining a partition across both arrays. A partition index i in array X uniquely determines a partition index j in array Y, such that the total number of elements in the left halves of both arrays equals the total number of elements in the right halves.

The primary invariant requires that the maximum element in the left partition of X is less than or equal to the minimum element in the right partition of Y, and conversely, the maximum element in the left partition of Y is less than or equal to the minimum element in the right partition of X. When this invariant is satisfied, the elements on the left side of the partitions strictly precede all elements on the right side in the combined sorted order.

To find the correct partition, the algorithm performs a binary search over the smaller array, which we denote as X without loss of generality. By binary searching on X, the search space is constrained to $[0, m]$. During each iteration, the algorithm calculates the midpoint i and the corresponding complementary point j. It then verifies the partition invariant. If the invariant fails, the search space is halved based on whether the left maximum of X is too large or too small. This structural approach ensures convergence to the exact median partition.

4 Complexity Analysis

The temporal complexity of this algorithm is bounded by the size of the search space. Because the binary search is conducted exclusively on the smaller array X, the maximum number of iterations is strictly proportional to the base-2 logarithm of its length. Consequently, the time complexity is $O(\log(\min(m, n)))$ in the worst case (Knuth, 1997).

The spatial complexity is $O(1)$. The algorithm requires only a constant number of integer variables to maintain the binary search bounds and partition indices. No auxiliary data structures are instantiated, and the input arrays are not mutated. This constant spatial footprint renders the algorithm optimal for memory-constrained execution environments.

5 Conclusion

This analysis establishes the mathematical and procedural correctness of the logarithmic-time algorithm for locating the median of two sorted arrays. By reformulating median selection as a binary search over partition indices, the algorithm strictly avoids linear-time merge operations. The resulting $O(\log(\min(m, n)))$ time complexity and $O(1)$ space complexity represent the theoretical optimum for this problem class.

References

[1] Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. (2009). Introduction to algorithms (3rd ed.). MIT Press.

[2] Knuth, D. E. (1997). The art of computer programming, volume 3: Sorting and searching (2nd ed.). Addison-Wesley.